

Interpretable CNNs: Visual Explanations for Image Classification on CIFAR-10

Hayam A. Rezk
School of Computing
Queen's University
25BXND@queensu.ca

Mohammed A. Anber
School of Computing
Queen's University
25tvcb@queensu.ca

Abstract—We implement Gradient-weighted Class Activation Mapping (Grad-CAM) [1] from scratch in PyTorch and evaluate it across three CIFAR-10 classifiers of increasing capacity: a custom three-block *BaselineCNN* (620K parameters), *ResNet18-scratch* (11.2M, random initialisation), and *ResNet18-pretrained* (11.2M, ImageNet weights). The implementation uses forward and full-backward hooks to capture feature maps and gradient tensors; importance weights are obtained by spatial averaging of gradients, followed by a ReLU-gated weighted sum, bilinear upsampling to 32×32 , and min-max normalisation. All three models were trained for 100 epochs on an NVIDIA RTX 5000 Ada GPU, reaching test accuracies of 87.3%, 93.3%, and 95.9% respectively. Spearman rank correlation against `pytorch-grad-cam` [8] yielded $r=1.00$ for each model, confirming mathematical equivalence. Both sanity checks of Adebayo et al. [2] are implemented: the data randomisation test passes cleanly for randomly-initialised models; the model randomisation test reveals a structural limitation discussed in detail. Remaining work includes a Grad-CAM++ [3] comparison and a dropout ablation on *BaselineCNN*.

Index Terms—Grad-CAM, CNN interpretability, CIFAR-10, transfer learning, sanity checks, saliency maps

I. INTRODUCTION

A trained CNN maps an input image to a class score but provides no direct account of which spatial regions drove that decision. In medical imaging, autonomous driving, and industrial inspection this matters concretely: a model achieving high accuracy for the wrong reasons will fail silently when the test distribution diverges from training. Spatially auditing a network's evidence is therefore a prerequisite for high-stakes deployment, not a post-hoc convenience.

This project builds Grad-CAM [1] from its defining equations rather than wrapping a library, because every step demands precise justification: why gradient of the logit rather than the activation weight directly; why spatial averaging rather than max-pooling for importance scalars; why the raw logit and not the post-softmax probability; what the final ReLU discards. These are not stylistic choices—each has a concrete mathematical consequence.

Three model variants on CIFAR-10 [7] isolate two independent effects: *BaselineCNN* vs. *ResNet18-scratch* isolates architectural depth; *ResNet18-scratch* vs. *ResNet18-pretrained* isolates the benefit of ImageNet representations. Three verifiable goals: (i) confirm correctness via Spearman $r > 0.95$ against the reference library; (ii) characterise how heatmap

resolution and semantic precision change with capacity and pretraining; and (iii) apply Adebayo et al. [2] sanity checks and report all findings honestly.

II. RELATED WORK

CAM. Zhou et al. [4] showed that a network with Global Average Pooling (GAP) before the classifier implicitly weights feature map channels by discriminative value. The constraint is strict: CAM requires the classification pathway to be a single linear layer after GAP, excluding *BaselineCNN* by construction.

Grad-CAM. Selvaraju et al. [1] replaced fixed weight projection with gradient signals, applicable to any differentiable architecture. Spatial averaging of $\partial y^c / \partial A^k$ produces α_k^c ; a ReLU-gated weighted sum yields the heatmap. Using the raw logit y^c rather than softmax p^c isolates the single-class signal cleanly, since softmax couples all scores through $\sum_j e^{y^j}$.

Guided Backpropagation and Sanity Checks. Springenberg et al. [6] produced sharp pixel-level maps by modifying the ReLU backward pass. Adebayo et al. [2] later showed these maps change negligibly under weight randomisation, directly motivating our sanity check evaluation.

Grad-CAM++ and ResNet. Chattopadhyay et al. [3] replace uniform spatial averaging with a pixel-wise weighted combination from a second-order Taylor expansion, targeting small or fragmented regions. He et al. [5] address degradation in deep networks via residual connections, which matter for Grad-CAM because gradient quality at the target layer depends on the faithfulness of the backward pass above it.

III. METHODOLOGY

A. Dataset and Preprocessing

CIFAR-10 [7] provides 60,000 RGB images at 32×32 pixels across ten classes (50,000 train, 10,000 test). Class balance is verified programmatically before any run. Training augmentation: `RandomCrop(32, padding=4)` and `RandomHorizontalFlip(p=0.5)`. Channel-wise normalisation uses $\mu=[0.4914, 0.4822, 0.4465]$, $\sigma=[0.2470, 0.2435, 0.2616]$ computed from the training split only. A sample grid is shown in Fig. 1.

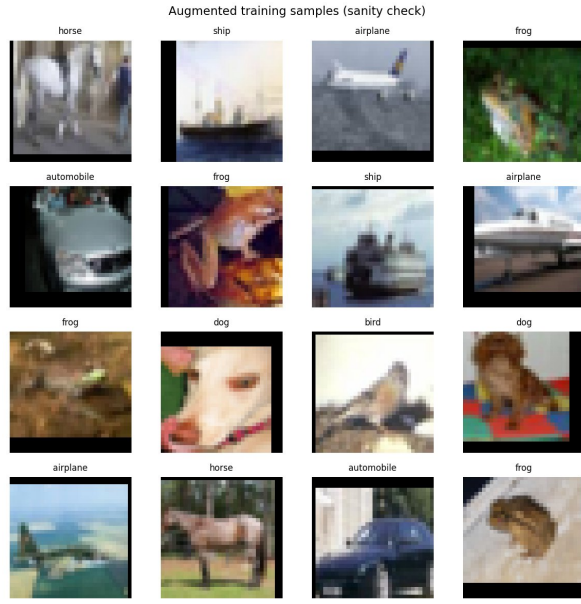


Fig. 1. Augmented training sample grid (16 images). Crop and flip effects confirm the pipeline prior to training.

B. Model Architectures

BaselineCNN. Three blocks of $\text{Conv}(k=3, p=1)\text{--BN--ReLU--MaxPool}(2)$; channels $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$; spatial $32 \rightarrow 16 \rightarrow 8 \rightarrow 4$. Classifier: Flatten--Linear(2048, 256)--ReLU--Dropout(0.5)--Linear(256, 10). Grad-CAM target: layer3 ($128 \times 4 \times 4$, 16 positions). Three MaxPool halvings on 32×32 set a hard ceiling of 16 activation levels independent of gradient quality.

ResNet18 variants. Standard ResNet18 [5] is adapted for 32×32 : the 7×7 stride-2 stem is replaced with $\text{Conv2d}(3, 64, k=3, s=1, p=1, \text{bias}=\text{False})$ and the initial MaxPool with $\text{nn.Identity}()$; the head becomes $\text{Linear}(512, 10)$. Grad-CAM target: layer4[-1] ($512 \times 4 \times 4$)—same spatial resolution but 512 channels yield a substantially richer weighted sum. For *ResNet18-pretrained*, weights are loaded from `ResNet18_Weights.IMAGENET1K_V1`; the replaced conv1 discards its pretrained weights while layer1–layer4 retain ImageNet initialisation.

C. Training Protocol

All models: SGD, momentum 0.9, weight decay 5×10^{-4} , lr 0.01, batch 128, 100 epochs, cosine annealing ($T_{\max}=100$). Global seed 42 fixed across all libraries.

D. Grad-CAM Implementation

GradCAM registers a forward hook capturing $A^k \in \mathbb{R}^{B \times C \times H \times W}$ and a `register_full_backward_hook` capturing $\partial y^c / \partial A^k$. Pipeline: (1) forward pass, hook captures A^k ; (2) determine target class (default: `argmax`); (3) extract raw logit y^c (not softmax score); (4) `score.backward()`, hook captures gradients; (5) $\alpha_k^c = \frac{1}{HW} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}$;

(6) weighted sum $\sum_k \alpha_k^c A^k$; (7) $\text{ReLU}(\cdot)$ to discard suppressive channels; (8) bilinear upsample to 32×32 ; min-max normalise. Hooks are removed in `__exit__` to prevent accumulation. Correctness is validated by Spearman r against `pytorch-grad-cam` on the same image and target class.

Both Adebayo sanity checks are implemented: `model_randomization_test()` cascades `nn.init.normal_` from output downward in five stages; `data_randomization_test()` trains a fresh model for 10 epochs on shuffled labels.

Code is available at <https://github.com/MohammedAnber/deeplearningp>.

IV. PRELIMINARY EXPERIMENTS AND RESULTS

A. Classification Performance

Table I and Fig. 2 summarise 100-epoch performance. Depth alone (*BaselineCNN* $\rightarrow$ *ResNet18-scratch*) adds +6.0 pp; pretraining adds +2.6 pp further.

TABLE I
TEST-SET PERFORMANCE AFTER 100 EPOCHS

Model	Acc. (%)	CE Loss
BaselineCNN	87.3	0.3825
ResNet18-scratch	93.3	0.2823
ResNet18-pretrained	95.9	0.1640

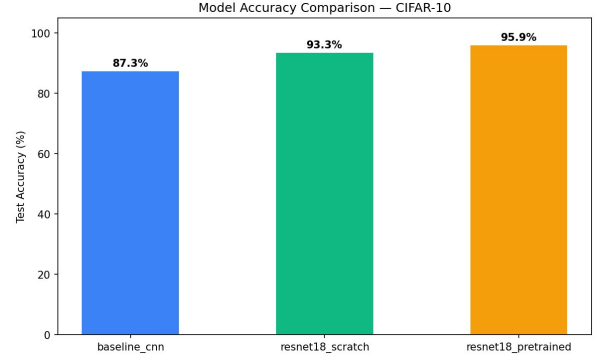


Fig. 2. Test accuracy comparison. Depth (+6.0 pp) and pretraining (+2.6 pp) gains are additive.

B. Training Dynamics

Fig. 3 shows training curves for all three models. *BaselineCNN* tracks train/val loss closely until epoch 30; thereafter the gap widens gradually. *ResNet18-scratch* opens the gap earliest: training loss collapses near zero by epoch 40 while validation plateaus at 0.28–0.30, reflecting memorisation ahead of generalisation. *ResNet18-pretrained* is most instructive: training saturates near 100% by epoch 20, yet validation loss continues decreasing through epoch 60—cosine annealing extracting genuine generalisation gains from pretrained representations after training is effectively saturated, a pattern we had not anticipated prior to running the experiment.

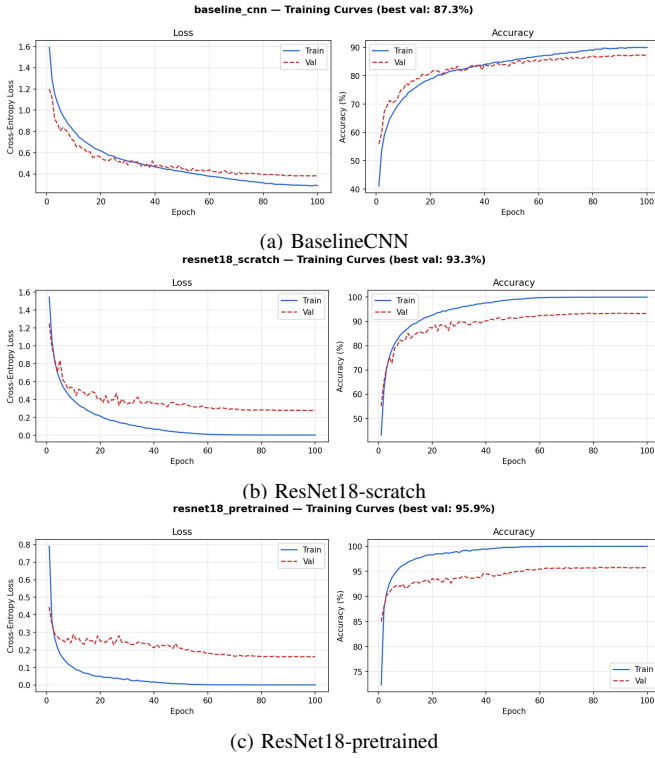


Fig. 3. Training curves. Solid: train; dashed: val.

C. Confusion Matrix Analysis

Cat/dog confusion persists across all three models. *BaselineCNN*: 119 cat→dog (cat: 729/1000) and 105 dog→cat. *ResNet18-scratch*: 72 and 62. *ResNet18-pretrained*: 48 cat→dog, but dog→cat remains at 62—unchanged from *ResNet18-scratch*—suggesting this direction is harder to resolve from the ImageNet prior. Fig. 4 shows the baseline and pretrained confusion matrices; the scratch matrix is intermediate in all entries.

D. Library Parity

TABLE II
SPEARMAN r VS. PYTORCH-GRAD-CAM

Model	r	Pass ($r > 0.95$)
BaselineCNN	1.00	Pass
ResNet18-scratch	1.00	Pass
ResNet18-pretrained	1.00	Pass

All three models achieve $r=1.00$: rank-identical heatmaps on the same test image and target class, confirming all steps reproduce the reference without systematic error.

E. Grad-CAM Heatmap Observations

Fig. 5 shows Grad-CAM grids for all three models. *BaselineCNN* maps are coarse and blocky: the 16-position 4×4 ceiling limits every upsampled map to block-level resolution, yet activation concentrates on object bodies for horse, truck, and frog. *ResNet18-scratch* maps are sharper; bird and deer

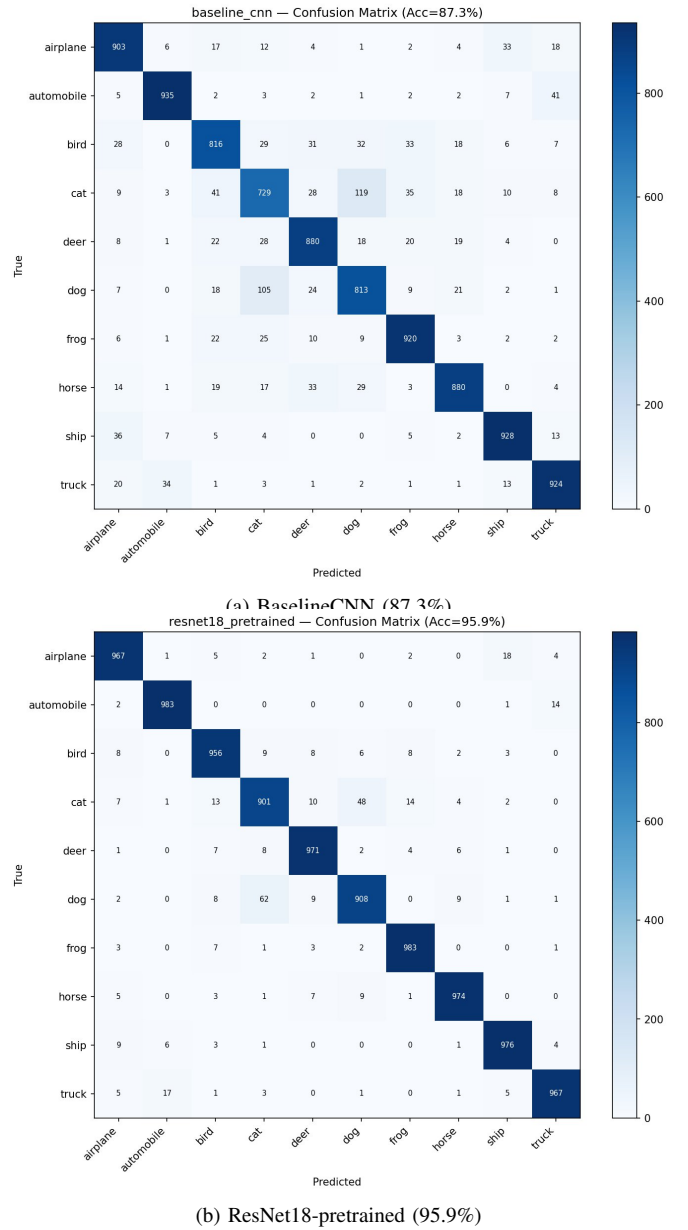
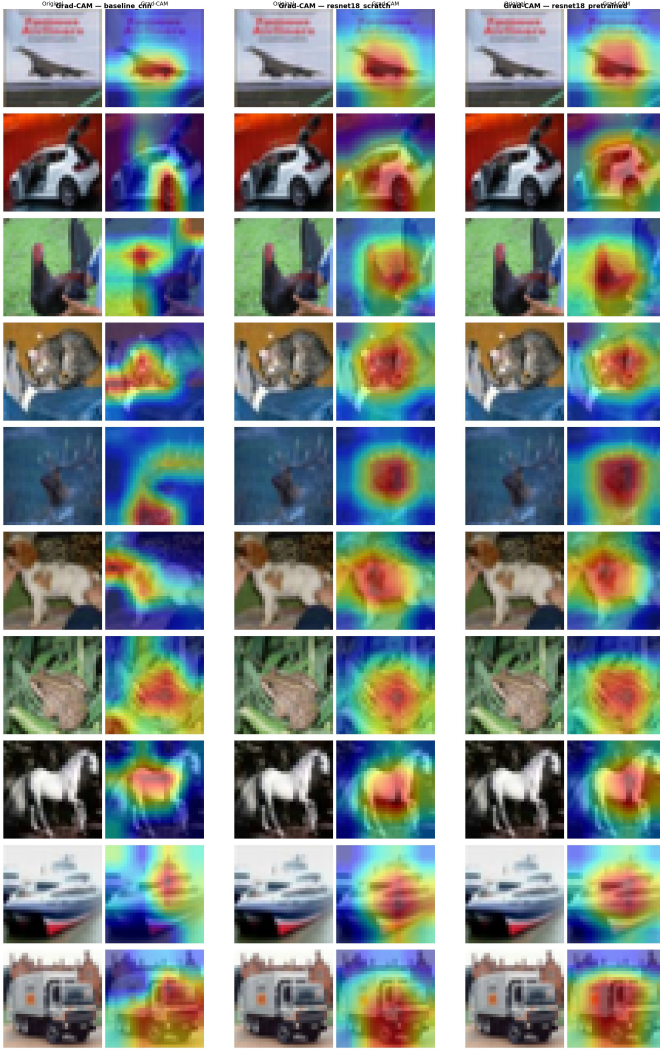


Fig. 4. Confusion matrices for baseline and pretrained models. Cat/dog is the dominant off-diagonal pattern.

show background diffusion consistent with learning from CIFAR-10 alone without large-scale pretraining. *ResNet18-pretrained* maps are tightest: ImageNet-tuned *layer4* retains part-selective character, concentrating on semantically meaningful regions.

F. Adebayo Sanity Checks

Data randomisation. Fig. 6 compares trained vs. shuffled-label heatmaps. For *BaselineCNN*, the trained model produces structured warm activation; the shuffled model is near-zero (unable to fit any valid boundary, α_k^c carries no class-specific information). *ResNet18-scratch* shows the same pattern. Both **pass**.



(a) BaselineCNN (b) ResNet18-scratch (c) ResNet18-pretrained

Fig. 5. Grad-CAM grids (original left, overlay right, jet colourmap) for all ten CIFAR-10 classes. Spatial precision increases with model capacity and pretraining.

ResNet18-pretrained produces an unexpected result: the shuffled-label model yields a warmer, more structured heatmap. The shuffled model was initialised from ImageNet weights and trained for only 10 epochs on random labels—insufficient to overwrite feature detectors in *layer1-layer4*. Adebayo et al. note this test is less interpretable for fine-tuned models; the result is **inconclusive** for this variant.

Model randomisation. Fig. 7 shows five cascade stages. *BaselineCNN* maps vary modestly; 4×4 resolution limits visual discriminability independent of gradient quality. *ResNet18-scratch* activation shifts without collapsing, suggesting some signal reflects feature map statistics surviving re-initialisation. *ResNet18-pretrained* heatmaps are near-identical across all stages: the cascade affects layers above the target hook while *layer1-layer3* retain ImageNet weights throughout. SSIM metrics across stages will be added in the final report.

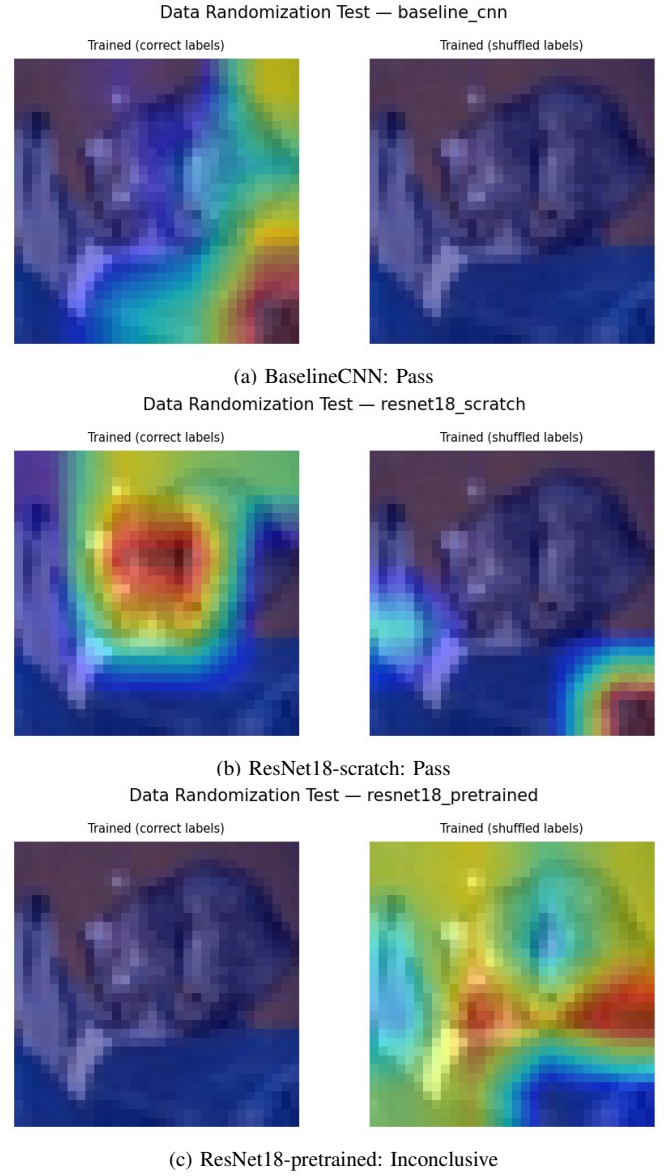


Fig. 6. Data randomisation. Left: trained (correct labels). Right: shuffled labels. Pretrained result is inconclusive due to retained ImageNet features.

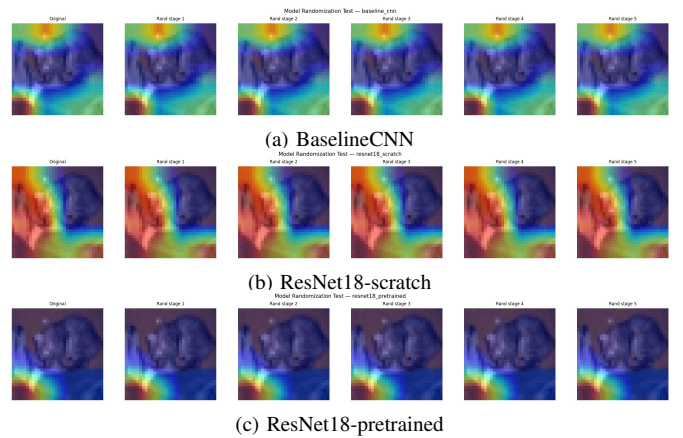


Fig. 7. Model randomisation (five cascade stages). ResNet18-pretrained maps are near-identical due to retained sub-target ImageNet weights.

V. PLANNED WORK AND TIMELINE

Week 3. (i) *Grad-CAM++ comparison*: generate maps using the `pytorch-grad-cam` Grad-CAM++ backend and compare qualitatively and quantitatively across all ten classes and three models. Small differences are expected at 32×32 . (ii) *Dropout ablation on BaselineCNN*: retrain with rates $\{0.0, 0.2, 0.4, 0.6, 0.8\}$ comparing accuracy and heatmap coherence to test whether regularisation pressure propagates to spatial evidence encoding.

Week 4. Final report: complete results table, pixel-wise SSIM metrics for model randomisation, Grad-CAM++ analysis, and unified interpretability discussion. Known risks: model randomisation sensitivity limitation is documented and will be quantified; Grad-CAM++ gains at 32×32 may be negligible.

VI. TEAM CONTRIBUTIONS

Hayam A. Rezk designed and implemented *BaselineCNN* and `build_resnet18()`—including the 3×3 stride-1 stem, `nn.Identity()` MaxPool, and `Linear(512, 10)` head—and built the full training infrastructure: `train_one_epoch()`, `evaluate()`, `check_initial_loss()`, cosine annealing scheduler, and checkpoint saving. Ran all three 100-epoch experiments on the RTX 5000 Ada GPU and generated all training curve plots. *Weeks 3–4*: dropout ablation; final results table.

Mohammed A. Anber implemented GradCAM from scratch (hook registration, eight-step pipeline, context manager); implemented `verify_against_library()`, `overlay_heatmap()`, `save_heatmap_grid()`, `model_randomization_test()`, `data_randomization_test()`, and `run_full_evaluation()`; drafted this report. *Weeks 3–4*: Grad-CAM++ comparison; SSIM sanity-check metrics; final analysis.

VII. USE OF AI TOOLS

We used Claude AI on three occasions.

First, during GradCAM development our backward hook stored $(1, C, H, W)$ rather than (C, H, W) . We had already formed the hypothesis that the leading dimension was the batch axis and `grad_out[0]` was correct, and asked Claude AI to confirm this is the documented behaviour of `register_full_backward_hook`. It confirmed our reading. The diagnosis, reasoning, and fix were ours before we asked.

Second, after the pretrained model produced an unexpected data randomisation result, we had already written our explanation—retained ImageNet features surviving 10 random-label epochs—and asked Claude AI whether Adebayo et al. address the fine-tuned case directly in the paper. It confirmed the paper flags this as a complicating factor.

Third, we used Claude AI to assist with $\text{\LaTeX}$ formatting and fixing compilation errors in the report source, including resolving layout issues.

REFERENCES

- [1] R. R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *Proc. ICCV*, 2017, pp. 618–626.
- [2] J. Adebayo et al., “Sanity Checks for Saliency Maps,” in *Adv. NeurIPS*, 2018, pp. 9505–9515.
- [3] A. Chattopadhyay et al., “Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks,” in *Proc. WACV*, 2018, pp. 839–847.
- [4] B. Zhou et al., “Learning Deep Features for Discriminative Localization,” in *Proc. CVPR*, 2016, pp. 2921–2929.
- [5] K. He et al., “Deep Residual Learning for Image Recognition,” in *Proc. CVPR*, 2016, pp. 770–778.
- [6] J. T. Springenberg et al., “Striving for Simplicity: The All Convolutional Net,” in *Proc. ICLR Workshop*, 2015.
- [7] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Tech. Rep., Univ. of Toronto, 2009.
- [8] J. Gildenblat et al., “pytorch-grad-cam,” GitHub, 2021.